

Figure 2: To create our datasets, we first generate sets of random propositions about two people. Each set is formatted with a template (SYNTH), rewritten into a story (PARA), and translated into Spanish (TRANS). We train probes to predict propositions from the easy SYNTH dataset, and test probes on the hard PARA and TRANS datasets.

Then, run the model $\mathcal{M}$ on the input T, and let the internal activations be $Z_0, \ldots, Z_{|T|-1}$, which we take to be the concatenation of pre-layernorm activations at every layer, so that $Z_k \in \mathbb{R}^{n_{\text{layers}} \times d_{\text{model}}}$. The task is to predict the set of propositions B from the internal activations $Z_0, \ldots, Z_{|T|-1}$.

Easy-to-hard evaluations To evaluate propositional probes we train our probes on simple inputs and test their generalization on hard, potentially adversarial, inputs. Prior works have used this easy-to-hard evaluation paradigm (Roger et al., 2023; Mallen & Belrose, 2023) to measure the usefulness of probes as monitoring methods. Specifically, we create three versions of datasets; the propositional probes are trained only using the simplest version (SYNTH), and evaluated on the heldout versions (PARA, TRANS). Further, in Sec. 6.2 we introduce additional distribution shifts during evaluation that adversarially changes the model's output behavior, and test if the probes can remain faithful.

Datasets We create three datasets of natural language passages of increasing complexity, namely SYNTH, PARA, and TRANS, each labelled with ground-truth propositions. To do so, we first generate random propositions, which are then formatted in a template to produce the SYNTH dataset, and augmented using GPT-3.5-turbo to produce diverse and complex PARA and TRANS datasets (Fig. 2).

In more detail, each input instance describes two people, each of whom has a name, country of origin, occupation, and a food they like. We model this information as a set of propositions such as $\mathsf{LivesIn}(\mathsf{Carol}, \mathsf{Italy})$. Formally, our closed world consists of four domains $\mathcal{D}_0, \ldots, \mathcal{D}_3$, which are sets of names, countries, occupations, and foods respectively. There are three predicates, LivesIn, WorksAs, and LikesToEat, each of which binds a name $E \in \mathcal{D}_0$ to an attribute from one of the three attribute domains $\mathcal{D}_1, \mathcal{D}_2$, or $\mathcal{D}_3$ respectively. Note that because of the one-to-one correspondence between predicates and attribute domains, the predicate of a proposition can be inferred by which domain the attribute is from. Our method utilizes this observation. In addition, for brevity we sometimes drop the predicate, e.g. $(\mathsf{Carol}, \mathsf{Italy})$ instead of $\mathsf{LivesIn}(\mathsf{Carol}, \mathsf{Italy})$.

To create the input context, we generate six random propositions about two people by sampling without repetition two values from each domain. These propositions are formatted in a template to produce the SYNTH dataset, rewritten using GPT-3.5-turbo into a short story (PARA), and then translated into Spanish (TRANS). See Fig. 2 for an example and Appendix A for details.

Models We use the Tulu-2-13b model (Ivison et al., 2023), an instruction-tuned version of Llama 2 (Touvron et al., 2023) with $n_{\text{layers}} = 40$ layers and $d_{\text{model}} = 5120$ embedding dimensions.

4 PROPOSITIONAL PROBES

In this section we describe the overall architecture of propositional probes. At a high level, propositional probes are composed of domain probes, one for each domain, and the outputs of the domain probes are composed to form propositions using the binding similarity metric. We construct domain probes in Sec. 4.1, compose them to form propositional probes in Sec. 4.2, but defer the construction of the binding similarity metric to Sec. 5. We make code available at https://github.com/jiahai-feng/prop-probes-iclr

Code

4.1 DOMAIN PROBES

For every domain, we train a domain probe that linearly classifies activations at individual token positions into either a value in the domain, or a null value $\perp$, indicating that none of the values is

represented. For example, Fig. 1 (left) shows the outputs of the name probe and the country probe. Formally, for every domain $\mathcal{D}_k$, we train a probe $P_k : \mathbb{R}^{d_{\text{model}}} \to \mathcal{D}_k \cup \{\perp\}$. We use the activation at a particular layer l as the input of the probe. We describe later how l is chosen.

We parameterize the probe with $|\mathcal{D}_k|$ vectors, $u_k^{(0)}, \dots u_k^{(|\mathcal{D}_k|-1)} \in \mathbb{R}^{d_{\text{model}}}$ and a threshold $h_k \in \mathbb{R}$. Each vector is a direction in activation space corresponding to a value in the domain. The classification of the probe is simply the value whose vector has the highest dot product with the activation, or the null value $\perp$ instead if all the dot products are smaller than the threshold. Formally,

$$P_k(Z) = \begin{cases} \arg\max_i u_k^{(i)} \cdot Z, & \text{if } \max_i u_k^{(i)} > h_k \\ \perp, & \text{otherwise} \end{cases}$$

To learn the vectors, we generate a dataset of activations and their corresponding values. Then, we set each vector to the mean of the activations with that input. Then, we subtract each vector with the average vector $\frac{1}{|\mathcal{D}_k|}\sum_i u_k^{(i)}$. This can be seen as a multi-class generalization of the so-called difference-in-means probes (Mallen & Belrose, 2023).

We collect this dataset of activations from the SYNTH dataset. However, this only provides context-level supervision: we know that the activations in the context collectively represent certain values in the domain, but we do not know which activations represent the value and which represent $\perp$. Thus, we have to assign the activations at each token position with a ground-truth label.

To do so, we use a Grad-CAM-style attribution technique (Selvaraju et al., 2017) similar to that used by Olah et al. (2018). Broadly speaking, we backpropagate through the model to estimate how much the activation at each layer/token position contributes towards the model's knowledge of the lexical information, which estimates the saliency of both the layer and token position.

The attribution results indicate that the middle layers at last token position are the most informative. We thus choose layer $l = 20$ (out of 40 layers). We discuss the attribution further in Appendix G.

4.2 PROPOSITIONAL PROBES

Our propositional probes compose the outputs of constituent probes with the binding similarity metric using a simple lookup algorithm. We first describe the binding similarity metric, and then describe the algorithm for propositional probes.

We expect the binding similarity metric to be greater for activations that are bound together than activations that are not. In the example in Fig. 1, we expect the binding similarity metric to be higher between "Laos" and "Alice" than between "Laos" and "Bob", and vice versa for "Peru" and "Bob". Concretely, the binding similarity metric $d(Z_s, Z_t)$ is a real-valued function that describes how strongly bound the activations Z_s and Z_t are to each other. In Sec. 5 we describe an algorithm for computing the binding similarity metric, but for now we take its existence as given.

Our propositional probes use the binding similarity metric to decide how the predicted attributes should be bound to the predicted entities. Specifically, we first identify all the names mentioned in the context with the name domain probe P_0. Then, for every other domain probe P_k, we identify the values it picks up in the context, and for each of these values we select the name with the highest binding similarity metric to compose together. The pseudocode is described in Alg. 1.

5 BINDING SUBSPACE

In this section, we present a Hessian-based algorithm for identifying the binding subspace, from which we construct the binding similarity metric used in the propositional probes. We first review background information on the binding subspace (Sec. 5.1), before describing our algorithm for identifying the binding subspace and constructing the binding similarity metric (Sec. 5.2). Lastly, we show that the resultant binding subspace is causally mediating (Sec. 5.3).

5.1 BINDING SUBSPACE BACKGROUND

The binding subspace arises from a decomposition of internal activations into lexical and binding vectors (Feng & Steinhardt, 2023). While researchers have long noted that language model acti-

Algorithm 1 Lookup algorithm to propose predicates

Require: Domain probes $\{P_k\}_k$ and binding similarity metric $d(\cdot, \cdot)$.

procedure PROPOSEPREDICATES($\{Z_s\}_s$)
  $N \leftarrow$ Subset of $\{Z_s\}_s$ for which P_0 is not $\perp$ ▷ Detect names
  for all $P_k, k > 0$ **do**
    $V \leftarrow$ Subset of $\{Z_s\}_s$ for which P_k is not $\perp$
    for all Z_v in V **do**
      $n \leftarrow \arg\max_{i \in N} d(Z_i, Z_v)$ ▷ Find best matching name
      Propose new proposition (n, v)
    end for
  end for
  return All proposed propositions
end procedure

vations often comprise of lexical vectors that linearly encode lexical concepts, Feng & Steinhardt (2023) recently observed that activations can also contain binding vectors that bind lexical concepts in one activation with lexical concepts in another.

Specifically, suppose the input describes two entities E_0, E_1 (e.g. "Alice", "Bob") with corresponding attributes A_0, A_1 (e.g. "Laos", "Peru") (see Fig. 1). Let Z_{E_k} and Z_{A_k} for $k = 0, 1$ be the activations for E_k and A_k respectively. Then, Feng & Steinhardt (2023) observed that the activations can be decomposed into lexical vector representations $f_E(E_k)$ and $f_A(A_k)$ of entities and attributes, and binding vectors $b_E(k)$ and $b_A(k)$,

$$Z_{E_k} = f_E(E_k) + b_E(k), \quad Z_{A_k} = f_A(A_k) + b_A(k),$$

so that interventions that modify $Z_{E_k} \leftarrow Z_{E_k} + b_E(k') - b_E(k)$ for $k' \neq k$, will switch the bound pairs (e.g. from $\{(E_0, A_0), (E_1, A_1)\}$ to $\{(E_0, A_1), (E_1, A_0)\}$), and likewise for Z_{A_k}.

However, the techniques in prior work could not identify the binding subspace (i.e. the subspace binding vectors live in), or the geometric relationship between the binding vectors $[b_E(k), b_A(k)]$. In particular, they relied on estimating differences in binding vectors $\Delta_E \triangleq b_E(1) - b_E(0)$, $\Delta_A \triangleq b_A(1) - b_A(0)$. To do so they computed the difference $Z_{E_1} - Z_{E_0}$ across 200 contexts with different values of E_0 and E_1, with the hope that the content dependent vectors f_{E_0}, f_{E_1} cancelled out (likewise for Δ_A). This enabled the bound pair switching intervention described in the preceding paragraph, but did not reveal how the binding vectors are geometrically related.

In this work, we hypothesize that there is low rank linear relationship between binding vectors that correspond to each other, i.e. there exists a low-rank binding matrix $\boldsymbol{H}$ under which related entity/attribute binding vectors have high norms, so that $b_E(k)^\top \boldsymbol{H} b_A(k')$ is high only when $k = k'$. We further posit that $\boldsymbol{H}$ is orthogonal to the subspaces spanned by the lexical vectors $f_E(E_k), f_A(A_k)$, so that $Z_{E_k}^\top \boldsymbol{H} Z_{A_{k'}} \approx b_E(k)^\top \boldsymbol{H} b_A(k')$. In Sec. 5.2 we describe a Hessian-based algorithm to identify $\boldsymbol{H}$, and construct a symmetric metric $d(Z_k, Z_l)$ that describes whether Z_k is bound to Z_l, which is used ultimately by propositional probes to compose domain values together.

5.2 HESSIAN-BASED ALGORITHM

To motivate the Hessian-based algorithm, we briefly discuss a straightforward approach to estimate the binding matrix $\boldsymbol{H}$ and a key shortcoming. A simple way to estimate $\boldsymbol{H}$ is to collect a dataset of model activations, together with ground-truth labels of which activations are bound together. Then, one can obtain $\boldsymbol{H}$ as either the result of bilinear regression, or with a more sophisticated loss involving causal interventions (Geiger et al., 2024)[1]. However, in either case, the estimated $\boldsymbol{H}$ is limited because it can only ever capture variations on binding vectors present in the initial dataset; in our setting where we train on a simple dataset and test on hard, diverse datasets, we need a method that captures structure inherently present in the model, and not just in the dataset.

At a high level, the Hessian-based approach approximates the model with its Hessian, which characterizes not just the model's behavior on binding vectors in the present input, but in all directions in

[1]We implement Distributed Alignment Search as a baseline in Sec. 5.3

$$\mathcal{E}_t = \left(r_t^{(\mathrm{ms})}, \Delta_t, g_{t+1}, e_t, \mathrm{ID}_t\right), \tag{18}$$

where $r_t^{(\mathrm{ms})}$ is the verified market state, Δ_t records the direction and magnitude of the proposed deferral, g_{t+1} is the counterfactual net advantage, $e_t \in \{0, 1\}$ indicates whether the proposal produced a non-zero executed intervention, and ID_t is a unique record identifier. GroundTrader also records the settlement time and uses experience records settled before the current decision for subsequent supervision.

For a new candidate, GroundTrader first constructs a set of comparable experience records settled before the current decision. The selected records share the same market regime, supervisory role, and one-period horizon. When at least three comparable records are available, the record analyst reviews the complete comparable set and cites the records supporting its assessment. GroundTrader verifies the cited identifiers and independently recomputes the historical success rate and mean counterfactual net advantage from the supplied records. The record-based assessment is validated when the cited evidence and reported statistics are consistent with the settlement history. Detailed comparability conditions and validation rules are provided in Appendix A. When fewer than three comparable records are available, GroundTrader uses a sparse fallback based on settled failures from executed interventions. An executed intervention with a non-positive counterfactual net advantage creates a temporary cooldown for the same transition direction. The next eligible deferral proposal in that direction can be withheld once using the corresponding settled record, after which the cooldown is cleared.

The record review returns one of three outcomes. The possible outcomes are AUTHORISE, WITHHOLD, and ABSTAIN. An AUTHORISE outcome retains the proposed deferral based on the matched historical records. A WITHHOLD outcome suppresses the proposal. A record-stage ABSTAIN leaves the coherence-stage DEFER recommendation unchanged. The proposal then enters the execution rule without a record-based override. Both AUTHORISE and ABSTAIN give $q_t = 1$. A WITHHOLD outcome gives $q_t = 0$, as defined in Section 3.2.2.

4. Experiments

4.1. Experimental Setup

We evaluate GroundTrader under three experimental settings. The main evaluation includes nine financial assets spanning seven equities and two cryptocurrencies: AAPL, AMZN, GOOGL, MSFT, TSLA, MRK, LLY, ETHUSD, and XRPUSD. For the main evaluation, historical price and news data are obtained from Financial Modeling Prep (FMP) [32]. For the 2020–2021 extended-period evaluation, the DIS, ENB, and ABT price series are obtained from Yahoo Finance [33], while their news records are drawn from the FNSPID-derived `VincentKriek/LLM-S-P-500-Dataset` [34,35]. The ETHUSD price series is obtained from frozen Yahoo `ETH-USD` daily market data [33], while its news records are drawn from the `crypto-news-coindesk-2020-2025` dataset [36]. For the cross-upstream evaluation, the AAPL, META, CRM, and NVDA price series are obtained from Yahoo Finance [33], while the news records are drawn from the `FelixDrinkall/financial-news-dataset` [37]. The corresponding upstream decision trajectories are generated using the official StockBench batch decision-agent component [38]. AAPL, AMZN, GOOGL, MSFT, and TSLA are evaluated from 1 June 2023 to 28 December 2023, covering 146 decision periods. ETHUSD is evaluated from 1 June 2023 to 31 December 2023, covering 214 periods. MRK and LLY are evaluated from 1 June 2022 to 31 May 2023, covering 251 periods, while XRPUSD is evaluated over the same calendar interval with 365 periods. The extended-period evaluation includes DIS, ENB, ABT, and ETHUSD over 2020–2021. DIS, ENB, and ABT are evaluated

from 2 January 2020 to 31 December 2021, covering 505 decision periods, while ETHUSD is evaluated from 1 January 2020 to 31 December 2021, covering 731 periods. The cross-upstream evaluation uses trajectories generated by the StockBench decision-agent workflow [38] for AAPL, META, CRM, and NVDA from 2 January 2020 to 31 December 2021, covering 505 decision periods. Table 1 summarizes the corresponding evaluation settings.

Table 1. Experimental assets and evaluation settings. Transaction costs are reported as one-way basis points (bp).

Evaluation	Asset Type	Assets	Evaluation Period	Decision Periods	Annual. Factor	Initial Capital	One-Way Cost (bp)
Main	U.S. large-cap equities	AAPL, AMZN, GOOGL, MSFT, TSLA	2023-06-01–2023-12-28	146	252	$10,000	10.0
Main	Cryptocurrency	ETHUSD	2023-06-01–2023-12-31	214	365	$100,000	1.5
Main	U.S. healthcare equities	MRK, LLY	2022-06-01–2023-05-31	251	252	$10,000	10.0
Main	Cryptocurrency	XRPUSD	2022-06-01–2023-05-31	365	365	$100,000	1.5
Extended	Equities	DIS, ENB, ABT	2020-01-02–2021-12-31	505	252	$10,000	10.0
Extended	Cryptocurrency	ETHUSD	2020-01-01–2021-12-31	731	365	$100,000	1.5
Cross-upstream	U.S. equities	AAPL, META, CRM, NVDA	2020-01-02–2021-12-31	505	252	$10,000	10.0

All methods are evaluated under the same long-only target-position setting, with the executed position constrained to $p_t \in [0, 1]$. The backtest follows a close-to-close accounting convention. The period-t closing observation defines the decision boundary, and the resulting position p_t is held over the subsequent t-to-$(t+1)$ interval. Its return is therefore evaluated using the next observed close. We assume zero slippage and apply the one-way transaction costs reported in Table 1. The 1.5 bp cryptocurrency cost is used as an idealized low-friction reference case for the controlled comparison rather than as a general estimate of real-world cryptocurrency execution costs. To avoid relying on this favorable cost assumption, the sensitivity analysis further evaluates ETHUSD and XRPUSD under one-way transaction costs of 5, 10, 20, 30, and 50 bp. Equity and cryptocurrency results are annualized using 252 and 365 periods per year, respectively. All trading decisions use only information available at decision time. For each decision date, the system receives at most 12 headlines published during the preceding seven calendar days and before the current decision time. The next-period market return becomes available only after the current position has been determined. It is subsequently used for portfolio evaluation and one-period counterfactual settlement, but never enters the current decision. This protocol ensures that all methods use the same decision-time information and prevents future market information from entering the supervisory process.

GroundTrader is evaluated on fixed action–reasoning trajectories generated by the upstream trading agent. In the main and extended-period evaluations, the trajectories are generated by FinAgent. The original actions, target positions, and supporting analyses are retained, while GroundTrader supervises the resulting position transitions before execution. The cross-upstream evaluation uses trajectories generated by the StockBench decision-agent workflow. For each decision period, StockBench provides a trading action, target position, and supporting rationale. GroundTrader processes these outputs through the same transition-level supervisory interface. Direct execution and GroundTrader supervision use the same stored upstream trajectory and the same deterministic base router within each paired comparison. The direct branch executes $p_t = p_t^{(\text{base})}$, whereas GroundTrader differs only through the supervisory execution weight w_t applied after routing. The base-routing trajectory is fixed throughout the evaluation, so each period-t proposal is settled against the same realized period-$(t+1)$ base position. In all evaluations, the